Explaining the alert triage model

How the model reaches each decision, what it can and cannot explain, and the mathematics underneath it. Release v20, frozen 2026-09-24.

The short answer

Yes, every decision this model makes can be explained, and exactly rather than approximately. The model is a **linear** classifier over **TF-IDF** text features. Its score is built by adding up one number per word or word-pair in the alert, then converting the total to a probability. So for any alert we can list which words and phrases pushed it toward a human analyst, which pushed it toward auto-close, and by how much. Those numbers add up exactly to the score.

What the model decides

The model answers one question: **is this alert a Benign Positive?** It does *not* choose between True Positive and False Positive. Both are grouped together as *not benign: a human should look*. That grouping is deliberate, and it sets the limit on what an explanation can claim. The model can explain why an alert does or does not look like the routine, expected activity analysts have closed before. It cannot explain why an alert is malicious.

Analyst disposition	Model class	Route	Alerts	Share
Benign Positive - Suspicious But Expected	0 benign	AI lane (auto-close)	25,963	80.1%
True Positive - Suspicious Activity	1 not benign	Human analyst	5,723	17.6%
False Positive - Incorrect Analytic Logic	1 not benign	Human analyst	580	1.8%
False Positive - Inaccurate Data	1 not benign	Human analyst	161	0.5%
Other / Undetermined / never dispositioned	not used	-	91,470	-

What an explanation can say, at a glance

Question a SOC manager asks	Can the model answer it?	How
Why was this alert auto-closed / sent to a human?	**Yes, exactly**	Per-term contributions that sum to the score (Section 3)
Which parts of the alert mattered most?	**Yes**	Rank terms by contribution; group them by CEF field (Section 4)
What does the model look for in general?	**Yes**	Global weights: the terms most associated with each lane (Section 4)
Have we seen alerts like this before, and what did analysts decide?	**Yes, with a small addition**	Nearest past alerts by cosine similarity, with their dispositions (Section 4)
How sure is it?	**Yes, statistically**	A calibrated probability, plus a threshold with a stated error bound (Section 6)
Is this a true positive or a false positive?	**No**	Not what it was trained on (Section 8)
Why is this activity malicious?	**No**	It learns associations from analyst decisions, not attacker behaviour

1. How an alert becomes a score

When Splunk ES raises a notable it becomes a SOAR container, and Apollo forwards the notable to the model. Seven steps then turn it into one number. The first five are deterministic text preparation. Nothing is learned in them, so they can be audited line by line.

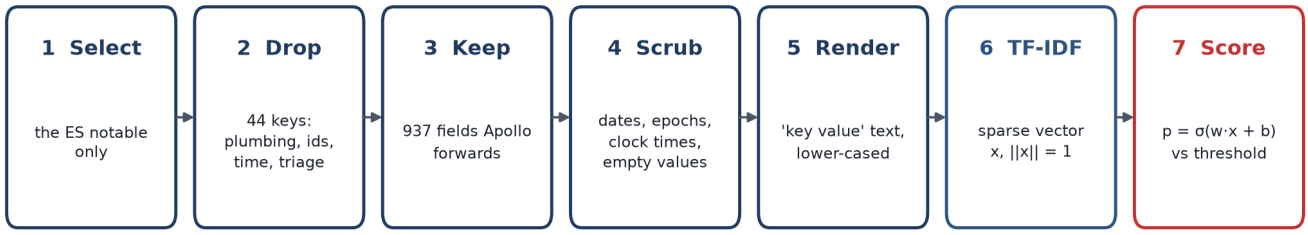


Figure 1. The preprocessing and scoring pipeline. Steps 1-5 come from config/preprocessing.yaml, which both training and serving read.

Step 1: read only what existed before the analyst did

A SOAR container holds up to four kinds of artifact, and only one of them existed before an analyst touched the alert. Reading any of the others would let the model see the answer:

Artifact	How it is identified	Used?
ES notable: the alert as it arrived	rule_id present	**yes**
Settings / metrics	carries SA_DISPOSITION	no: that field *is* the label
soar_event=*	incident_closure, sa_decision, ...	no: written during triage
Slack / escalation note	reply_count, snooze_count	no: exists only once escalated

Steps 2-4: remove what would mislead

44 fields in seven groups are dropped before the model sees anything. Each group has a stated reason:

Group	Keys	Why it is removed
splunk plumbing	6	internal Splunk storage handles; no bearing on the alert
per event identifiers	6	unique to one alert — a memorised handle on a single training row, never a pattern that transfers
timestamps	10	encode WHEN, not WHAT. The not-benign base rate ran 6.5% -> 25.3% across the training window, so a date predicts the window and nothing beyond it
date parts	8	the same, pre-split by Splunk into components
console urls	4	per-alert links back into ES or SOAR; unique handles
triage state	8	LEAKAGE — describes ES-side triage, which is set AFTER the alert arrives and after an analyst has touched it
serving alignment	2	present on only one side of the SOAR/Apollo boundary. Leaving them in makes the training string and the serving string differ for the same alert (mean 0.042, max 0.38).

Timestamps are also **scrubbed from every value**: ISO dates, epochs, bare years and clock times. This matters because the share of not-benign alerts rose from 6.5% to 25.3% over the training year. A date therefore predicts when an alert happened, not what it is. An early fit ranked *2026 04* among its strongest signals. After scrubbing, the model can only learn from *what* fired and *what it fired on*. Of the remaining fields, only the 937 that Apollo has been seen to forward are kept, so the model is trained on exactly what it will receive in production.

Step 5: one string per alert

Whatever survives is written out as key `value` pairs and lower-cased. Pairs rather than bare values let a two-word phrase bind a field to its value: `severity low` and `urgency low` stay distinguishable. An excerpt of a real alert, with the customer and user masked:

```
clientname customer-a cat initialaccess ... destinationusername user@customer-a.org detectionsource
officeatp entity_type alert indicator 216.81.249.98 ... rule_name ts - defender - def07 - new high
severity alerts security_domain threat severity high sourceaddress 216.81.249.98 title a potentially
malicious url click was detected urgency medium ...
```

That string is the model's only input: one column, `text_cef`. There is no per-customer history, no rule track record and no time of day. Steps 6 and 7 (TF-IDF and scoring) are explained conceptually in Section 2 and mathematically in Section 5.

2. Why this model can be explained exactly

Many machine-learning models are black boxes, and explaining them means approximating them, with tools such as SHAP or LIME estimating what the model probably did. This model needs no approximation, because of its shape:

$$p = \sigma(z), \qquad z = b + \sum_j w_j x_j \tag{1}$$

- $\mathbf{x_j}$ is the TF-IDF value of term *j* in this alert: how prominent the word or word-pair is here, discounted by how common it is across all alerts. It is zero for terms that do not appear.
- $\mathbf{w_j}$ is the weight the model learned for term *j*. Positive means alerts containing it were more often sent to humans; negative means more often closed as Benign Positive.
- **b** is the intercept, the starting point before any evidence.
- **σ** is the logistic function, which turns any total *z* into a probability between 0 and 1.

The total *z* is a plain sum. Each term's share of it, its **contribution** $c_j = w_j x_j$, is exact. The contributions plus the intercept add back to *z* with nothing left over, and *z* fixes the score. An explanation built from contributions is therefore what the model computed, not a story told about it afterwards.

$$z = b + c_1 + c_2 + \cdots + c_k, \qquad c_j = w_j x_j \tag{2}$$

Reading a contribution

Contributions are in **log-odds**, the natural unit of the logistic model. They have a direct interpretation: a contribution c multiplies the odds that the alert needs a human by e^{c}.

$$\frac{p}{1-p} = \exp\left(b + \sum_j c_j\right) = e^{b} \cdot e^{c_1} \cdot e^{c_2} \cdots e^{c_k} \tag{3}$$

Contribution c	Odds multiplied by e^{c}	Plain reading
+0.153 (defender)	1.165	odds of needing a human up 16.5%
+0.095 (severity alerts)	1.100	up 10.0%
-0.145 (org)	0.865	down 13.5%
-0.190 (98)	0.827	down 17.3%

Because the threshold is also a point on the same scale, the routing rule can be stated in the same units. The shipped threshold p = 0.007093 corresponds to z = -4.94. **An alert is auto-closed only if the evidence for and against adds up to below -4.94**, which requires a lot of evidence that it is routine.

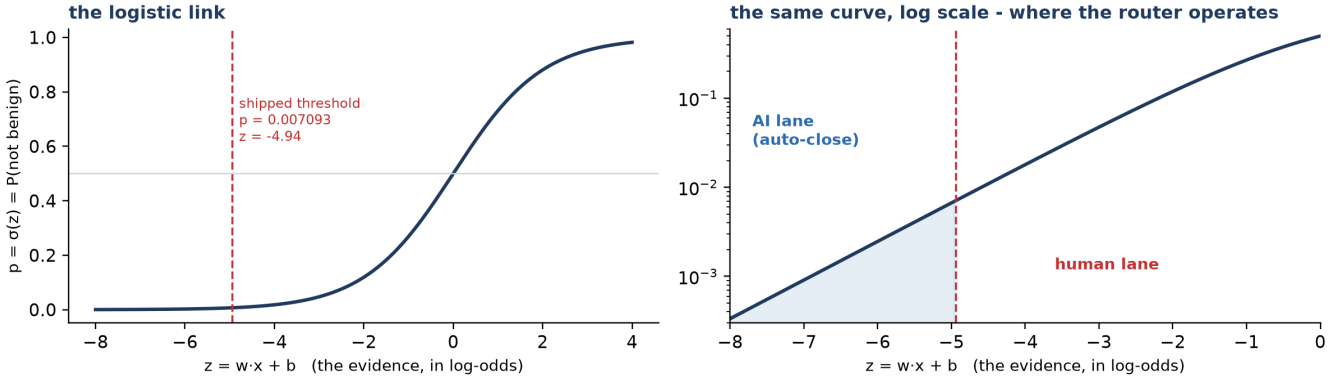


Figure 2. The logistic link from evidence z to score p. The router works in the far-left tail: an alert must be very confidently routine before it is auto-closed.

3. Explaining one real alert

Container **prod-soar:229895**, a Microsoft Defender detection (DEF07, *a potentially malicious URL click was detected*). The analyst disposition was **True Positive**. This example is taken from the project's documented walkthrough on an earlier release, fitted on the 22,699 alerts closed before this one, so the model had never seen it. The arithmetic is the same in the current release; only the learned weights differ.

- **Arrives:** 5 CEF artifacts, 203 keys. Four artifacts are discarded (label, closure, and two Slack escalation records).
- **Kept:** the ES notable, 84 keys, rendered to one string of 285 words.
- **Vectorised:** 255 non-zero terms out of 168,679 in that release's vocabulary. Every other entry of x is zero and contributes nothing.

Term	tf-idf x	weight w	contribution w·x	Direction
`defender`	0.102	+1.50	+0.153	toward human
`severity alerts`	0.117	+0.81	+0.095	toward human
`search incidentid`	0.055	+1.45	+0.080	toward human
`sourceaddress 216`	0.083	+0.74	+0.062	toward human
`malicious`	0.076	+0.76	+0.058	toward human
98	0.124	-1.54	-0.190	toward AI
`org`	0.073	-1.97	-0.145	toward AI
`potentially malicious`	0.145	-0.59	-0.085	toward AI
`detectionsource officeatp`	0.066	-0.96	-0.064	toward AI
the other 246 terms + intercept			-1.491	
total evidence z			**-1.527**	**p = σ(z) = 0.1785**

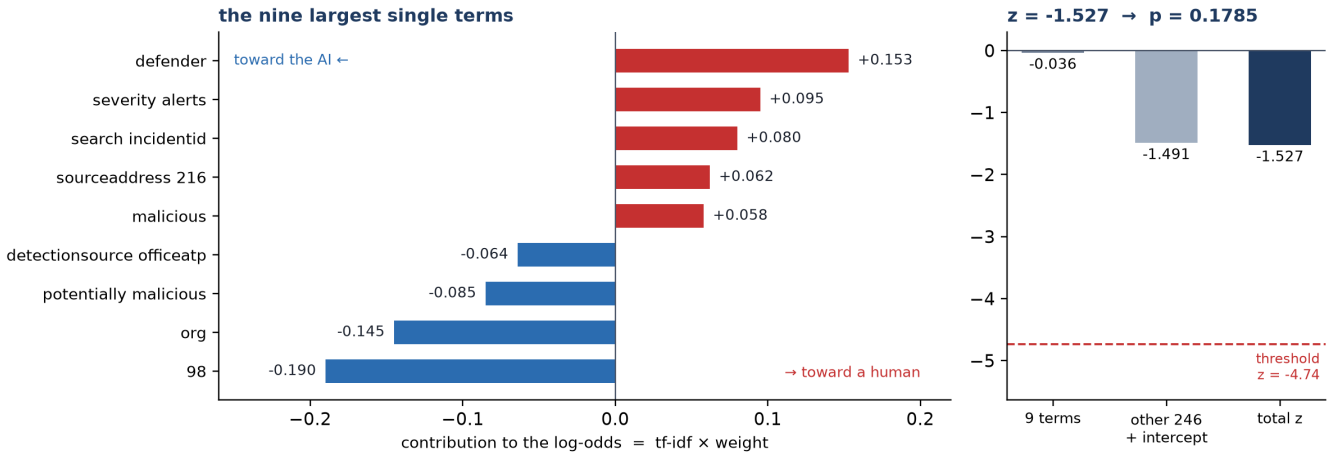


Figure 3. Left: the nine single terms with the largest contributions. Right: how all contributions add to the score. The remainder is computed so that the total equals the documented score.

What the explanation tells an analyst

- **Decision:** score 0.1785 is far above that release's threshold of 0.0087 (z = -4.74), so the alert goes to a human. That was correct: it was a true positive.
- **Pushing toward a human:** *defender*, *severity alerts* and *malicious*, the vocabulary of the high-severity Defender rule, plus *search incidentid* and the source address.
- **Pushing toward auto-close:** *org*, *98* and *detectionsource officeatp*, tokens that appear often on routine Office ATP alerts.
- **What it does not say:** the model is not 18% sure this is malicious. It says the alert is *not confidently routine*, and that is the only judgement it is asked to make. At a textbook 0.5 cut-off this alert would have been missed; the low threshold catches it.

Explanations surface quirks

Note 98 and `sourceaddress 216`. The tokeniser splits 216.81.249.98 into octets, so the model reads *parts* of an IP address as separate words. Explanations expose quirks like this, which is a reason to show them. An analyst can see when a decision leans on something that should not matter.

4. Four kinds of explanation the model supports

4.1 Local: why this alert

This is the per-term breakdown above. The production scoring API already returns it: every `/predict` response includes `top_terms`, the twelve terms with the largest absolute contributions, along with the score, the threshold, the margin, every field that was dropped and why, and how many of the alert's words the model has never seen. A word the model has never seen contributes exactly zero, so a high unknown-word count means the explanation is thin.

Mock-up: an explanation card for prod-soar:229895

DECISION	**HUMAN LANE**: send to an analyst
Score	P(not benign) = **0.1785** against threshold 0.0087 (margin +0.1698)
Toward a human	defender +0.153 · severity alerts +0.095 · search incidentid +0.080
Toward auto-close	98 -0.190 · org -0.145 · potentially malicious -0.085
By field	*proposed, Section 4.2*: e.g. rule fields, user fields, source address, each with its summed contribution
Similar past alerts	*proposed, Section 4.4*: the 5 nearest alerts, their similarity and how analysts closed them
Coverage	255 terms recognised; the API also reports words it has never seen
Read as	*not confidently routine*, not *18% likely malicious*

Rows 1-4 and 7 come from today's /predict response; rows 5-6 are the proposed additions.

4.2 By field: which part of the alert

Managers usually think in CEF fields, not tokens. Because the input is written as `key value` text, each token's position tells us which field it came from. Contributions can therefore be summed per field: *rule_description +0.31, destinationUserName -0.22, severity +0.08*. This is a small addition to the API, and the sums still add up to z. One caveat is that a two-word phrase can span the end of one field and the start of the next, so those phrases have to be split or shown under both fields.

4.3 Global: what the model looks for

Sorting all weights w_j gives the model's overall vocabulary of risk: the terms most associated with each lane across all alerts. This is the right artifact for a model review board. The fitted text is concentrated in a few fields:

CEF field	On % of alerts	% of fitted text
rule_description	98.4	11.0
savedsearch_description	95.4	10.7
orig_rule_description	78.6	8.9
source	100.0	5.4
search_name	100.0	5.4
rule_title	100.0	4.9
orig_rule_title	78.9	4.3
rule_name	100.0	4.0
contributing_events_search	20.8	1.5
destinationUserName	78.9	1.1

The top ten fields make up 57.4% of all fitted text and the top fifty make up 82.0%. The 976 fields that appear on fewer than 1% of alerts make up 3.7%. So the model mostly learns *which detection fired* and *whom and what it fired on*.

4.4 By precedent: alerts like this one

This is the explanation analysts trust most, and the model's representation supports it directly. Every alert is a unit-length TF-IDF vector, so the similarity of two alerts is just their dot product, the **cosine similarity** (Section 5.3). For a new alert we can retrieve the k most similar past alerts and show how analysts closed them: "*The 5 most similar alerts in the last 90 days (similarity 0.91-0.96) were all closed Benign Positive by the MDR desk.*" This is an addition, not something the model does today. It reuses the same vectoriser and needs an index of recent alerts with their dispositions.

> **Recommendation**
> Show managers the **precedent view** first, the **field view** second, and keep the **term view** for analysts and model reviews. All three come from the same numbers; they differ only in how they are grouped.

5. The mathematics in depth

5.1 Tokens and n-grams

The rendered string is lower-cased, accents are stripped, and it is split into tokens with the pattern `\b\w\w+\b`: runs of two or more letters or digits. Single characters and punctuation are discarded, which is why an IP address becomes its octets. The features are every **unigram** (single token) and every **bigram** (pair of adjacent tokens). So `severity high` yields *severity*, *high* and *severity high*. With `min_df = 1`, any term seen even once in training is kept. This gives a vocabulary of hundreds of thousands of terms: 635,806 when fitted on the 22,699-alert training split.

5.2 TF-IDF: turning text into numbers

TF-IDF gives each term a weight that rises with how prominent the term is in this alert and falls with how common it is across all alerts. A word that appears on every alert, such as *rule_name*, tells the model little; a rare one tells it a lot. For term t in alert d, with n training alerts:

$$\mathrm{tf}(t,d) \;=\; 1 + \ln \mathrm{count}(t,d) \qquad \text{(sublinear; 0 if absent)} \tag{4}$$

$$\mathrm{idf}(t) \;=\; \ln \frac{1+n}{1+\mathrm{df}(t)} + 1 \tag{5}$$

$$v_t \;=\; \mathrm{tf}(t,d)\cdot \mathrm{idf}(t), \qquad x_t \;=\; \frac{v_t}{\sqrt{\sum_s v_s^2}} \tag{6}$$

- **Sublinear tf** (4): a term repeated 11 times counts 1 + ln 11 = 3.4, not 11. Alerts that list the same event repeatedly (*a user account was created* eleven times) would otherwise be dominated by the repetition.
- **Smoothed idf** (5): df(t) is the number of training alerts containing t. The +1 terms prevent division by zero and keep terms that appear everywhere at a small positive weight.
- **L2 normalisation** (6): every alert vector has length 1, so a long alert and a short alert are on the same scale. One consequence for explanations: a term's x_t, and so its contribution, is smaller in a long alert, because its share of a longer text is smaller.

Worked example. Three toy alerts, vectorised here with the production settings. The table shows alert 2, `rule_name defender high severity alerts severity high`. *severity* and *high* each appear twice, so their tf is 1 + ln 2 = 1.693. *severity* and *rule_name* appear in every toy alert, so their idf is the minimum, 1.0.

term t	count	tf	df	idf	tf·idf	x_t (normalised)
`high`	2	1.693	1	1.693	2.867	**0.484**
`alerts`	1	1.000	1	1.693	1.693	**0.286**
`alerts severity`	1	1.000	1	1.693	1.693	**0.286**
`defender`	1	1.000	1	1.693	1.693	**0.286**
`defender high`	1	1.000	1	1.693	1.693	**0.286**
`high severity`	1	1.000	1	1.693	1.693	**0.286**

term t	count	tf	df	idf	tf·idf	x_t (normalised)
rule_name defender	1	1.000	1	1.693	1.693	**0.286**
severity	2	1.693	3	1.000	1.693	**0.286**
severity alerts	1	1.000	1	1.693	1.693	**0.286**
severity high	1	1.000	1	1.693	1.693	**0.286**
rule_name	1	1.000	3	1.000	1.000	**0.169**

Vector length before normalising: $\sqrt{\Sigma v^2} = 5.918$. Every value in the last column was checked against scikit-learn's output to 12 decimal places.

5.3 Cosine similarity: the basis for precedents

Because every vector has unit length, the similarity of two alerts is just a dot product. It ranges from 0 (no terms in common) to 1 (the same text):

$$\cos(\mathbf{x}_a, \mathbf{x}_b) \;=\; \frac{\mathbf{x}_a \cdot \mathbf{x}_b}{\|\mathbf{x}_a\|\,\|\mathbf{x}_b\|} \;=\; \sum_t x_{a,t}\, x_{b,t} \tag{7}$$

In the toy set, the two *multiple user creations* alerts, which differ only in the actor's name, have similarity **0.779**. The Defender alert and a user-creation alert have **0.089**.

5.4 The classifier: logistic regression

The estimator is scikit-learn's `SGDClassifier` with `loss = log_loss`. That is logistic regression, fitted by stochastic gradient descent instead of a batch solver. It models the probability that an alert is not benign as:

$$P(y = 1 \mid \mathbf{x}) \;=\; \sigma(\mathbf{w}^{\top}\mathbf{x} + b) \;=\; \frac{1}{1 + e^{-(\mathbf{w}^{\top}\mathbf{x} + b)}} \tag{8}$$

Equivalently, the log-odds are linear in the features: $\ln(p / (1 - p)) = \mathbf{w}\cdot\mathbf{x} + b$. That linearity is what makes equations (2) and (3) exact.

5.5 What training minimises

The weights are chosen to minimise the average **log loss** (cross-entropy) over the training alerts, plus an **L2 penalty** that keeps weights small:

$$J(\mathbf{w}, b) \;=\; \frac{1}{n}\sum_{i=1}^{n}\left[-y_i \ln p_i - (1 - y_i)\ln(1 - p_i)\right] \;+\; \frac{\alpha}{2}\,\|\mathbf{w}\|_2^2 \tag{9}$$

- The log loss is small when the model gives high probability to what analysts actually decided, and large when it is confidently wrong.
- **$\alpha = 10^{-5}$** sets the penalty strength. It was chosen by an out-of-fold sweep. The penalty stops the model from staking huge weights on terms seen on only a handful of alerts. Weights, and so contributions, stay moderate and comparable.
- **The classes are not reweighted.** Balancing the 80/20 split was measured and cost both ranking quality and auto-close coverage, so the probabilities reflect the true base rate.

5.6 How SGD finds the weights

Stochastic gradient descent visits the training alerts one at a time, in shuffled order, and nudges the weights after each one. For logistic loss, the gradient of one alert's loss with respect to z is just the prediction error (p_i - y_i), which gives a very readable update:

$$\mathbf{w} \leftarrow (1 - \eta_t \alpha)\,\mathbf{w} \;-\; \eta_t (p_i - y_i)\,\mathbf{x}_i, \qquad b \leftarrow b - \eta_t (p_i - y_i) \tag{10}$$

- If the analyst said *not benign* ($y = 1$) and the model said 0.2, the error is -0.8. Every term present in that alert has its weight *raised* in proportion to its TF-IDF value. A benign alert scored too high lowers them. Over 32,427 alerts, each weight settles where it best agrees with the analysts' decisions on the alerts that contain that term.
- The (1 - $\eta_t\alpha$) factor is the L2 penalty: every step shrinks all weights slightly toward zero.
- The step size follows scikit-learn's *optimal* schedule, $\eta_t = 1 / (\alpha(t + t_0))$, which starts large and decays as training proceeds.

Averaging. The release uses `average = True` (averaged SGD). The weights shipped are the mean of the weight vector over all updates, not the last value:

$$\bar{\mathbf{w}} \;=\; \frac{1}{T} \sum_{t=1}^{T} \mathbf{w}_t \tag{11}$$

Averaging cancels the jitter of single-alert updates and lands close to the exact logistic-regression solution, at a fraction of the cost. For explanations, this means the weights are stable: they do not depend much on which alert happened to be visited last. Training stops when the loss improves by less than 10^{-6} for 20 consecutive passes. The shuffle seed is fixed (7), so a release can be rebuilt exactly.

5.7 How this relates to SHAP

SHAP is the most common explainability method for black-box models. For a linear model whose features are treated as independent, SHAP values have a closed form:

$$\phi_j \;=\; w_j \left(x_j - \mathbb{E}[x_j]\right) \tag{12}$$

The only difference from our contribution $c_j = w_j x_j$ is the reference point. SHAP measures each term against the *average* alert, and the contributions measure it against an empty alert. Both are exact and both sum to the score. Running SHAP here would add cost and approximation and no new information.

6. How sure is it? The threshold and the evidence

6.1 Choosing the threshold with a stated error bound

The model only outputs a score. **The threshold is the one setting a person chooses**, and it is chosen statistically rather than by eye. Sort a recent *calibration* window of alerts by score and consider auto-closing the lowest n. If k of them were in fact not benign, the true error rate in the AI lane is uncertain, so we use its **Clopper-Pearson exact upper confidence bound**:

$$U(k, n) \;=\; B^{-1}(1-\delta;\; k+1,\; n-k) \tag{13}$$

Here B^{-1} is the inverse Beta distribution and 1 - δ = 95% is the confidence level. The threshold is the score below which the *widest* lane still satisfies U(k, n) ≤ 1%. The method is exact rather than a normal approximation, because the counts here are small and often zero, which is where approximations fail.

$$k = 0: \quad U(0, n) = 1 - \delta^{1/n} \;\le\; 0.01 \;\;\Leftrightarrow\;\; n \;\ge\; 299 \tag{14}$$

In plain words: to claim at 95% confidence that fewer than 1% of auto-closed alerts are mistakes, you need at least **299 auto-closed alerts with zero mistakes**, and more if any mistakes occur. The bound is set by the evidence rather than by a safety factor.

Shipped operating point	Value
Threshold on P(not benign)	0.007093 (z = -4.94)
Rule	Clopper-Pearson upper bound ≤ 1% at 95% confidence
Trained through / calibrated on / validated on	2026-06-02 / 2026-06-03 .. 2026-07-28 / 2026-07-29 .. 2026-09-23
Measured on the validation window	6.38% of alerts auto-closed, 6 of them not benign

6.2 The trade-off: coverage against misses

Allowing a few mistakes moves the line past the handful of not-benign alerts that score unusually low, and buys a lot of coverage. Out-of-fold over twelve months (27,023 alerts, 5,872 not benign):

Misses allowed	Threshold	Auto-closed	Alerts in AI lane	Not-benign rate in AI lane
0	0.001524	1.65%	444	0.0%
1	0.001536	1.68%	452	0.221%
2	0.001544	1.71%	461	0.434%
5	0.002069	3.0%	811	0.617%
10	0.002448	3.91%	1,055	0.948%
20	0.003656	6.28%	1,697	1.179%
50	0.006603	11.23%	3,033	1.649%

The thresholds are small because the region where the model is *certain* is narrow, not because the model is timid. On recent traffic, the single lowest-scoring not-benign alert sets the zero-miss line:

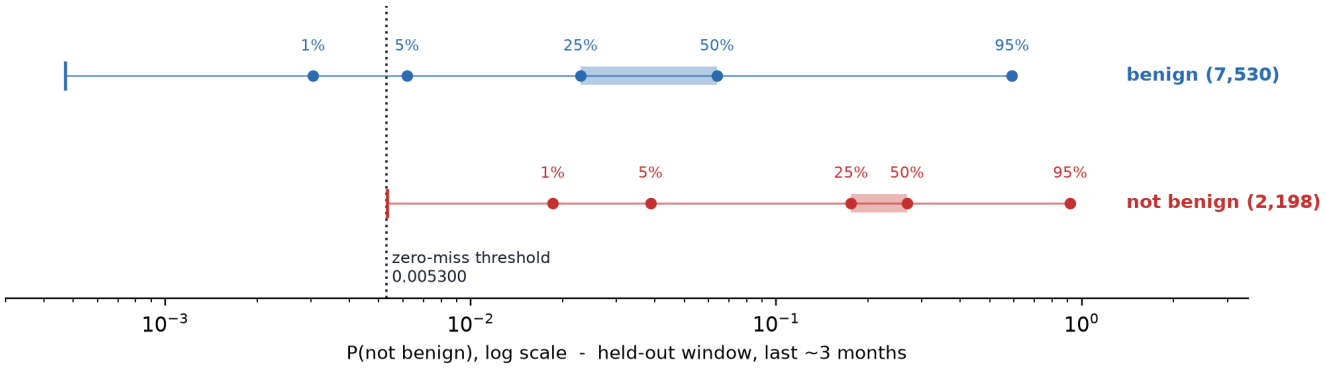


Figure 4. Score percentiles for benign and not-benign alerts on the held-out window (last ~3 months). Bars span the 25th-50th percentiles; ticks mark the minimum. The populations overlap widely; the router uses only the far-left tail.

6.3 Is the score a real probability?

A score is *calibrated* if, among alerts scored around 20%, about 20% turn out not benign. Log loss tends to produce calibrated scores without extra correction. Split by score into fifths:

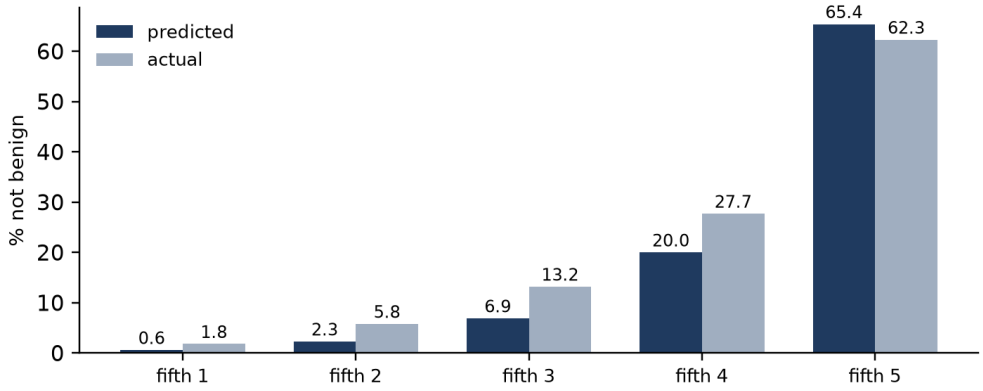


Figure 5. Predicted vs actual not-benign rate by score fifth, out-of-fold (manifest of release v20).

The top fifth is well calibrated (65.4% predicted, 62.3% actual). The bottom fifths under-predict: alerts scored 0.6% are not benign 1.8% of the time. **Read low scores as a ranking, not a literal probability.** This is exactly why the threshold comes from measured error rates (6.1) and not from the score itself.

6.4 Ranking quality

$$\text{ROC AUC} = P(s(\text{not benign}) > s(\text{benign})) = \frac{1}{n_P n_N} \sum_{i \in P} \sum_{j \in N} \mathbb{1}[s_i > s_j] \tag{15}$$

$$\text{AP} = \sum_{k} (R_k - R_{k-1}) P_k \tag{16}$$

P and N are the not-benign and benign alerts, s the score, and R_k, P_k the recall and precision at the k-th threshold.

ROC AUC 0.854: pick one not-benign and one benign alert at random, and 85.4% of the time the not-benign one scores higher. **PR AUC (average precision) 0.6893**, against a base rate of

21.73%, which is what random guessing would score. On the most recent, harder three months the held-out ROC AUC is 0.809. The model ranks well, but it does not separate the two classes cleanly, which is why it is used to clear the obvious routine alerts and not to make final calls.

7. What explanations from this model do not mean

An honest explanation states its limits. These should travel with any explanation shown to analysts or customers.

- **Association, not cause.** A positive weight means analysts more often sent alerts containing that term to the human lane. It is not evidence that the term is malicious. The model reflects past analyst decisions.
- **The labels are noisy.** Different closing desks disagree by 15.3 percentage points on the same rule at the same customer. The model learns the average decision, and no explanation can be more consistent than the labels it was trained on.
- **Duplicated text splits the credit.** The rule description appears in up to three fields (rule_description, savedsearch_description, orig_rule_description). Correlated terms share credit somewhat arbitrarily, so read the field total rather than any single term.
- **It memorises entities.** Usernames, email domains and IP octets carry real weight. That is part of its accuracy, a privacy consideration for anything that displays explanations, and a reason explanations can look odd (the *98* example).
- **Unseen words count for nothing.** A new customer, rule or host contributes zero until the next retrain. New customers' alerts therefore score near the base rate and go to humans, which is the safe default.
- **The base rate drifts.** The not-benign share rose from 6.5% to 25.3% over the training year. A threshold set on one period does not carry over to another, so it must be recalibrated on recent traffic before each deployment.
- **Confidence is not distance from the threshold.** A score of 0.004 against a threshold of 0.007 is a confident auto-close; a score of 0.45 against the same threshold is not a confident human-lane call. Near 0.5 the model is about as good as a coin flip.

8. The TP / FP / BP question

SOC managers asked for explanations of *true positive*, *false positive* and *benign positive*. The current model makes one cut, Benign Positive against everything else, for three reasons:

- **It is the routing question.** Only Benign Positives are safe to auto-close. True and false positives both need a human action: a response, or a fix to the detection.
- **False positives are rare.** 741 alerts in a year (2.3%), across two sub-types and many rules. That is too few to learn a reliable third class, and a weak FP class would make the BP explanations worse.
- **The clean class is benign.** Benign Positive - Suspicious But Expected means one thing, *we have seen this and it is fine*, and that is what the weights learn.

If TP-versus-FP explanations are needed, the sound design is a **second-stage model** applied only to human-lane alerts, trained to separate True Positive from False Positive. It would have the same linear form, so it would be explainable in exactly the same way. FP is largely a property of the *detection rule* (incorrect logic, inaccurate data), so rule-level FP rates over time may answer the managers' question better than any per-alert model.

9. Recommendations

#	Recommendation	Effort	Value
1	Show the **explanation card** on every routed alert: decision, score vs threshold, top terms each way, unknown-word count. The data already exists in the /predict response.	Small: UI only	High
2	Add **field-level attribution**: sum contributions by the CEF field each token came from (Section 4.2).	Small: API	High for managers
3	Add **precedents**: the 5 most similar recent alerts by cosine similarity, with their dispositions (Section 4.4).	Medium: needs an index	Highest for analysts
4	Publish a **global term report** per release: top weights each way, flagging entity-like terms (IP octets, usernames).	Small: notebook cell	Model governance
5	**Recalibrate the threshold** on recent traffic before each deployment, and report the Clopper-Pearson bound with it.	Existing: Stage 7	Safety
6	If TP vs FP is required, scope a **second-stage model** and rule-level FP reporting (Section 8).	Medium	Answers the original ask

Appendix: the shipped configuration

Setting	Value
Input	one text column: the ES notable as 'key value' text
Fields used	937 (forwarded by Apollo); 44 dropped; timestamps scrubbed
Tokeniser	\b\w\w+\b, lower-case, accents stripped
n-grams	1-2 (unigrams and bigrams)
TF	sublinear: 1 + ln(count)
IDF	smoothed: ln((1+n)/(1+df)) + 1
Normalisation	L2 (unit-length vectors)
min_df	1
Estimator	SGDClassifier, log_loss (logistic regression)
Penalty	L2, α = 1e-5
Averaging	averaged SGD (average = True)
Stopping	tol 1e-6, 20 passes without improvement, max 10,000 passes
Class weights	none (80 / 20 left as is)
Training data	32,427 alerts, 3 SOAR servers, 12 months to 2026-09-23
Scores	P(not benign); auto-close when below the threshold

Sources: models/v20/manifest.json, config/preprocessing.yaml, 02-train-model.ipynb (saved outputs), and ../triage/machine-learning/README.md (worked example). This document is generated by docs/build_explainability_pdf.py.